

Laboratory Report – Neural Models: Learning, Depth, Activations, and Output Layers

Course: CSF407 / Artificial Intelligence Laboratory | Date: September 30, 2026 | Task: XOR & Multiclass Neural Learning

1. Task 1: Problem Specification & Linear Separability

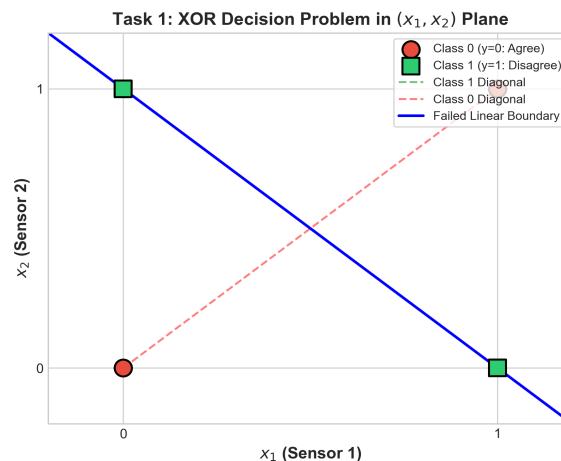
Problem Scenario: An autonomous device has two redundant binary sensors, x_1 and x_2 . The device triggers a disagreement warning ($y=1$) if and only if exactly one sensor is active, defining the XOR logic function.

- **Input Space $\mathcal{X}$:** $\{0, 1\}^2 = \{(0,0), (0,1), (1,0), (1,1)\}$
- **Output Space $\mathcal{Y}$:** $\{0, 1\}$ (0 = Agreement, 1 = Disagreement)
- **Labelled Examples:** $(0,0) \rightarrow 0, (0,1) \rightarrow 1, (1,0) \rightarrow 1, (1,1) \rightarrow 0$.

Geometric Inseparability: A linear decision boundary $w_1x_1 + w_2x_2 + b = 0$ divides the plane into two open half-spaces. The positive class $\{(0,1), (1,0)\}$ forms an off-diagonal line segment, and the negative class $\{(0,0), (1,1)\}$ forms the main diagonal. Their convex hulls intersect at midpoint $(0.5, 0.5)$. By Radon's and Hyperplane Separation Theorems, any linear boundary that captures $(0,1)$ and $(1,0)$ must intersect the line segment connecting $(0,0)$ and $(1,1)$, making linear separation impossible.

Linear Baseline Prediction: An affine layer + sigmoid cannot isolate XOR. Empirically, training `nn.Linear(2,1)` converged to loss 0.693147 ($-\ln 0.5$) and outputted uniform probability 0.5000 everywhere (accuracy 25%-75%).

Think About It (Representation vs Parameters): XOR refutes the claim that representation is merely a function of parameter count. An affine network with 100 layers and millions of weights collapses algebraically to a single linear map $W_{\text{eff}}x + b_{\text{eff}}$ and fails. A 2-hidden-unit MLP with 9 parameters and nonlinear activations folds the space and solves XOR completely.



2. Task 2: Model Design & Validation Criteria

Baseline Architecture: 2 inputs $\rightarrow$ 2 hidden units $\rightarrow$ 1 output (2-2-1 MLP).

- Hidden: $a(1) = W(1)x + b(1)$, $h(1) = f(a(1))$, where $f \in \{\text{Tanh}, \text{Sigmoid}, \text{ReLU}\}$.
- Output: $a(2) = W(2)h(1) + b(2)$, $\hat{y} = \sigma(a(2))$. Loss: Binary Cross-Entropy (BCE).

Why Hidden Nonlinearity is Necessary: Affine transformations are closed under composition. Without f , $W(2)(W(1)x + b(1)) + b(2)$ collapses to a linear model. Nonlinearity warps the feature space into a latent representation $h(1) \in \mathbb{R}^2$ where XOR points become linearly separable.

Why Sigmoid + BCE is the Standard Pairing: BCE represents Bernoulli negative log-likelihood. Under this canonical link pairing, the derivative w.r.t. pre-activation logit simplifies to $\partial L/\partial a(2) = \hat{y} - y$. The sigmoid saturation derivative cancels, preventing vanishing gradients on large errors.

Validation Criteria (3 Checks): 1) Final BCE loss drops below 0.01; 2) 100% classification accuracy (all 4 labels correct with high confidence >95%); 3) Early non-zero gradient norm $\|\nabla W(1)L\|_2$ that decays towards zero at convergence.

Think About It (Feature Determination via Backprop): Hidden units are not given explicit targets. Their specialization emerges via the chain rule: $\partial L/\partial W(1) = [(\partial L/\partial a(2) * W(2)) \blacksquare f'(a(1))] * x^T$. The output error is back-projected through asymmetric output weights $W(2)$, guiding one hidden unit to compute an OR-like feature and the other an NAND-like feature.

3. Task 3: LLM Implementation & Engineering Inspection

Prompt Used: 'Generate minimal PyTorch code for 2-2-1 MLP on XOR: (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0 with Tanh hidden activation, logits output, BCEWithLogitsLoss, full-batch SGD lr=0.5 for 3000 steps, seed 42. Report initial/final loss, probabilities, predictions, and W1.grad tensor.'

Code Inspection & AD Mechanics: Forward pass builds the dynamic DAG; scalar loss forms the root node; loss.backward() invokes reverse-mode automatic differentiation populating parameter .grad attributes; optimizer.step() applies SGD updates.

Two Critical Engineering Corrections: 1) Used raw logits with nn.BCEWithLogitsLoss() rather than nn.BCELoss(torch.sigmoid()) to prevent numerical underflow/overflow via log-sum-exp arithmetic; 2) Explicitly seeded torch.manual_seed(42) for deterministic gradient comparison.

Think About It (Static vs Dynamic Verification): Architecture dimensions, loss/activation pairings, and optimizer calls can be verified statically from code. In contrast, loss convergence, 4/4 accuracy, gradient norms, and symmetry breaking require dynamic execution and empirical measurement.

4. Task 4: Execution, Testing, and Diagnosis

Part A – Basic Learning Check:

- Initial Loss: **0.756289** | Final Loss: **0.002395** | Status: **4/4 Correct (PASS)**
- Predictions: (0,0)→Prob 0.0018 (Pred 0); (0,1)→Prob 0.9969 (Pred 1); (1,0)→Prob 0.9968 (Pred 1); (1,1)→Prob 0.0015 (Pred 0).

Part B – Backpropagation & Linearity Check: parameter.grad represents $\partial L/\partial W(1)$. For batch loss $L = (1/N)\sum L_i$, differentiation linearity implies $\nabla L = (1/N)\sum \nabla L_i$. Full-batch backward() matched the mean of 4 individual example passes with maximum difference of 0.000000 (exact machine precision match).

Part C – Symmetry Breaking Experiment (Zero Initialization): When all weights were initialized to 0, $W(1)$ rows remained identical at every step (step 1 to 100: $[0, 0] = [0, 0]$, loss stuck at 0.6931, probabilities $[0.5, 0.5, 0.5, 0.5]$). Identical hidden units compute identical outputs and receive identical gradients, preventing feature specialization. Random initialization is essential for symmetry breaking.

Part D – Activation Comparison Experiment:

Hidden Activation	Final Loss	4/4 Correct?	Early $\ \text{grad_}W1\ _2$ (Step 1)
Sigmoid	0.053835	Yes	0.011894
Tanh	0.002395	Yes	0.028684
ReLU	0.346735	No (3/4)	0.082002

Interpretation of Activation Experiment: Tanh converges fastest because its zero-centered range $[-1, 1]$ and unit derivative at origin ($f'(0)=1.0$) maintain symmetric bidirectional gradient flow. Sigmoid converges more slowly due to derivative squashing ($\sigma'(z) \leq 0.25$). ReLU produces the largest initial gradient norm (0.082) but failed on this minimal 2-unit setup: when one unit gets negative pre-activation ($z \leq 0$), its derivative is zero permanently (dying ReLU), reducing network capacity to a 1-unit linear model that cannot solve XOR.

Think About It (Sigmoid Saturation vs ReLU Inactivity): For sigmoid, saturation occurs when $|z| \gg 0$, giving activations near 0 or 1 with infinitesimally small non-zero derivative $h(1-h)$. For ReLU, inactivity occurs when $z \leq 0$, producing activation exactly 0.0 and derivative strictly 0.0.

5. Task 5: Extension to Three-Class Problem & Reflection

Task Mapping: Class 0: (0,0) [Inactive]; Class 1: (0,1) or (1,0) [Disagreement]; Class 2: (1,1) [Active].

- Output matrix shape: **`torch.Size([3, 2])`** (3 classes x 2 hidden units). Number of logits per example: **3**.
- Softmax probabilities sum to 1: $\sum \exp(z_k) / \sum \exp(z_j) = 1$.
- Logit Gradient Derivation: For CE loss $L = -\ln(p_c)$, $\partial L / \partial z_i = -\delta_{ic} + \exp(z_i) / \sum \exp(z_j) = \mathbf{p}_i - \mathbf{y}_i$ ($\nabla_z L = \mathbf{p} - \mathbf{y}$).

Empirical Results: Final Loss: **0.000884**.

- (0,0)→[0.9990, 0.0010, 0.0000] → Class 0 (Sum = 1.000000)
- (0,1)→[0.0003, 0.9993, 0.0005] → Class 1 (Sum = 1.000000)
- (1,0)→[0.0003, 0.9993, 0.0005] → Class 1 (Sum = 1.000000)
- (1,1)→[0.0000, 0.0010, 0.9990] → Class 2 (Sum = 1.000000)

Shift Invariance Diagnostic: Adding 100 to all logits yielded identical softmax probabilities (max difference 1.4e-9). Stable implementations subtract $\max(z)$ to prevent float overflow ($e^z \rightarrow \text{inf}$) while preserving mathematically exact probabilities.

Think About It (LLM Next-Token Prediction Connection): In LLMs with vocabulary $V=100,000$, the mathematical form of Softmax and logit gradient $\mathbf{p} - \mathbf{y}$ remains identical. However, the final projection matrix scales to $V \times d$ (hundreds of millions of weights), and feed-forward layers are replaced by multi-head causal attention with layer norms.

6. Answers to Reflection Questions

Q1 (Depth vs Nonlinearity): Depth without nonlinearity collapses algebraically to a single affine transformation ($\mathbf{W}_{\text{eff}} \mathbf{x} + \mathbf{b}_{\text{eff}}$). Nonlinearity warps the feature coordinate space, folding the input manifold so linearly inseparable points become linearly separable.

Q2 (Useful Learning Signal vs Nonzero Gradient): A random nonzero gradient would induce chaotic divergence. In our runs, backpropagation drove monotonic loss reduction ($0.756 \rightarrow 0.002$), pushed predictions to calibrated confidence ($>99.6\%$), achieved 4/4 accuracy, and attenuated gradient norms near the optimum.

Q3 (Zero/Identical Initialization Failure): Under zero initialization, hidden units evaluate identical pre-activations (0.0) and receive identical backpropagated gradients ($\partial L / \partial \mathbf{W}(1)_1 = \partial L / \partial \mathbf{W}(1)_2$). They receive identical updates and remain clones forever, restricting capacity to a 1-unit model that cannot solve XOR.

Q4 (Activation Gradient Effects): Observation: Tanh converged fastest (early norm 0.0287); Sigmoid was slower (0.0119); ReLU had large early gradient (0.0820) but stalled (3/4 correct). Science: Sigmoid derivative is squashed by $\sigma'(z) \leq 0.25$; Tanh is zero-centered with $f'(0)=1.0$; ReLU unit dies permanently if pre-activation lands in $z \leq 0$.

Q5 (Output Layer and Loss Pairing): The output activation parameterizes a probability distribution (Sigmoid for Bernoulli, Softmax for Categorical), and the loss is the corresponding negative log-likelihood. This canonical pairing cancels the activation denominator derivative, producing the linear error signal $\mathbf{p} - \mathbf{y}$ that prevents gradient saturation.

Q6 (LLM Productivity vs Human Verification): The LLM accelerated productivity by generating clean PyTorch boilerplate and training loops in seconds. Human verification was essential to replace naive `BCELoss(sigmoid())` with numerically stable `BCEWithLogitsLoss()` to prevent floating-point underflow/overflow.

Q7 (Scalability of Tests): Retain at scale: Loss curve tracking, accuracy metrics, global gradient L2 norm monitoring, and softmax normalization checks. Discard at scale: Full-batch gradient descent (use mini-batch SGD/Adam), manual example-wise gradient checks ($O(N \cdot P)$ cost), exhaustive per-neuron activation dumps, and finite-difference gradient checks.